ChatGPT

Enterprise HRMS Backend Architecture Proposal

Current Architecture Foundation

The existing HRMS backend is built on a single PostgreSQL database using multiple schemas for logical separation (e.g. `tenancy`, `iam`, `hr_core`, `workflow`, `dms`). This **modular schema design** already provides a good foundation for multi-tenancy and separation of concerns. For example, the `tenancy` schema holds tenant (client) information and plans, `iam` manages user accounts, roles, and permissions (RBAC), `hr_core` contains core employee data and organizational structure, `workflow` supports approval workflows, and `dms` handles document metadata/storage. All tenant-specific data is tagged by tenant ID, ensuring that each customer's data stays isolated in queries. The current system is essentially a **modular monolith** – a single deployment with clear domain schemas – which we will leverage and extend rather than replace. This foundation allows us to introduce new modules in an organized way that aligns with the existing structure.

Design Objectives: The new design must be scalable, maintainable, and secure. Key goals include:

- **Scalability:** Ability to handle growing users/tenants and high load by scaling horizontally and using efficient resource utilization.
- **Modularity & Separation of Concerns:** Each major HR function should be encapsulated in its own module or service with minimal coupling between domains [1]. This ensures changes in one area (e.g. payroll) have little impact on others, and each component can be developed and scaled independently.
- **Multi-Tenancy:** Continue to enforce strict tenant data isolation. Every authorization check and data query must be **tenant-aware** (e.g. "is user X an admin *for tenant Y*?") [2]. The same user may have roles in multiple tenants, so role assignments and permission checks will be scoped per tenant to prevent any cross-tenant data access.
- **RBAC Security:** Extend the existing role-based access control to cover new modules. Define new roles/permissions as needed (e.g. Recruiter, Payroll Manager, Line Manager, Employee self-service user) and ensure **no cross-tenant or unauthorized access** – one bug in RBAC could leak another tenant's data [3]. The auth layer should remain maintainable even as roles grow.
- **Extensibility:** The architecture should easily accommodate the full set of new HRMS features and future additions. A **modular microservice approach** (or well-structured modular monolith) will allow adding or modifying HR functions without overhauling the entire system [4] [5].
- **Reliability & Fault Tolerance:** The system should gracefully handle failures. Use asynchronous processing for long-running tasks, implement proper error handling and retries, and design for high availability (e.g. clustering and backups for the database, load-balancing services).
- **Enterprise Best Practices:** Implement comprehensive logging and monitoring (with traceability across services), robust authentication/authorization, and thorough auditing. Ensure the system can be debugged and audited easily (for compliance or troubleshooting), and support is in place for tracing requests end-to-end.

With these goals in mind, the following sections propose a scalable backend design that integrates all required HRMS features while building upon the current architecture.

Proposed Architecture Overview

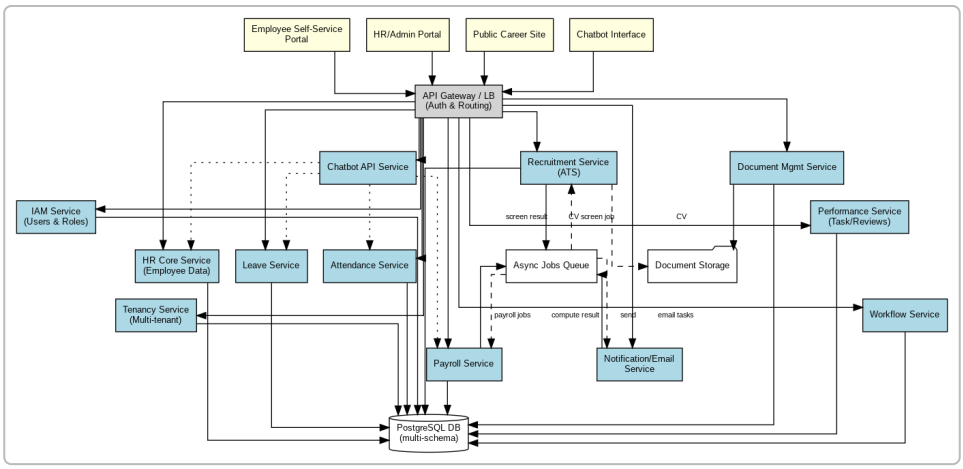

Figure: Proposed microservices-oriented architecture for the HRMS backend. The system is organized into multiple domain-driven services (modules) that interact through RESTful APIs and asynchronous messages. An API Gateway (or load balancer + routing layer) sits in front of these services to provide a unified interface, handle authentication, and route requests to the appropriate service. Each service corresponds to a functional module (Recruitment, Attendance, Payroll, etc.) and has its own schema in the multi-tenant PostgreSQL database (preserving the existing schema separation by domain). This modular approach follows HR technology best practices: **each microservice ("module") encapsulates a specific HR function** (recruitment, payroll, leave management, etc.) and exposes a well-defined API [5]. The services remain loosely coupled and highly cohesive – interacting only via APIs or message queues – which ensures that the overall system is easier to maintain and scale.

In the proposed design, core cross-cutting services like **Identity & Access Management (IAM)** and **Tenancy** remain central. The IAM service will issue and verify authentication tokens (e.g. JWTs) and enforce RBAC rules, while the Tenancy module provides tenant context (e.g. lookup tenant configurations, subscription limits, etc.) for all other modules. These foundational services can be factored out as standalone microservices or libraries used by other services, ensuring a single source of truth for users, roles, and tenant info. All incoming API calls first go through the gateway where the user's token is checked (using the IAM service) and tenant context is identified, after which the request is forwarded to the appropriate module's endpoint.

Each **domain service** (Recruitment, Leave, Attendance, Payroll, Performance, DMS, etc.) contains the business logic and data for that domain. They communicate primarily via synchronous REST API calls through the gateway. For example, the Self-Service Portal UI might call the "Leave Service" API to submit a leave request, or the Recruitment module might call the DMS service API to save a candidate's uploaded CV. In cases where one service needs data from another, they should use clearly defined interfaces (e.g. the Payroll service calling the Attendance service for an employee's hours). Direct cross-service calls can be done via REST internally or gRPC, but to maintain loose coupling it's preferable to use an **event-driven approach** for many interactions. We introduce an **asynchronous message queue (bus)** for events and background jobs: e.g. the Recruitment service can publish an event "Candidate Hired" which is consumed by the HR Core service (to create an employee record) and by the IAM service (to provision a user account). Likewise, heavy processing tasks are offloaded to background workers via the queue – for instance, a CV uploaded in recruitment triggers a "CV Screening" job processed by an AI/ML service worker, and the results are posted back when ready, or monthly payroll calculation jobs are queued to be computed by a worker thread. This event-driven pattern improves responsiveness and decouples services, enhancing scalability and fault tolerance.

The **shared PostgreSQL database** will continue to be used, with a new schema for each major new module (e.g. a `recruitment` schema for ATS, `attendance` for time logs, `payroll` for salary records, etc.). Each service exclusively owns its tables (no direct cross-schema table access without going through the service's logic) – effectively treating each schema as the domain's data store. This approach allows us to maintain data integrity with single DB transactions within a service, while still logically separating concerns. (In the future, if scaling demands, each service could be moved to its own database or database user – but initially, schemas in one DB simplify integration while enforcing namespace separation [6].) The multi-tenant design remains: every table in each schema includes a tenant identifier, and queries are always scoped to `WHERE tenant_id = X` (often automated by the data access layer). This guarantees **no cross-tenant data mixing**, satisfying a core security requirement [7]. We will also utilize **Row-Level Security** or a global tenant filter in each service's ORM to add an extra guard against accidental cross-tenant access.

Common utilities like caching and file storage are also depicted in the architecture. A distributed cache (e.g. Redis) can be used for caching frequent lookups (such as permission checks, reference data, or session tokens) to reduce database load. The Document Management Service (DMS) will use a **file storage** solution (like AWS S3 or an on-prem file server) for storing binary files (documents, images, videos) – the database will only store metadata and references. This is important for scalability, as large files are better kept out of the relational DB.

Finally, **cross-cutting concerns** are handled in a standardized way across all services. We will implement **centralized logging** with a correlation ID that propagates through services to trace each user request end-to-end [8]. For example, if a user's self-service action touches multiple services, all log entries for that action will carry the same request ID for easy debugging. We will also enforce consistent error handling, input validation, and authentication on every service boundary (potentially using shared middleware packages for consistency). In summary, this architecture provides a blueprint where each module is independent yet interoperable through well-defined interfaces and a shared foundation, yielding a system that is **flexible, scalable, and maintainable** in the long run [4].

Below, we dive into each proposed module, outlining its responsibilities, data design, APIs, and integration points with the rest of the system.

Domain Modules Design

1. Recruitment & ATS Module

Responsibilities: This module will handle end-to-end recruitment, functioning as an Applicant Tracking System (ATS). It covers creating job postings, receiving and managing applications, automated resume screening, scheduling interviews, and tracking candidates through the hiring pipeline until onboarding. It aims to eliminate the manual recruitment process (a key business challenge) by automating and centralizing hiring workflows.

Data Model: A new schema (e.g. `recruitment`) will contain tables such as:

- **JobPosition:** holds job details (title, department, description, requirements, status, etc.) and tenant_id. HR users can create positions which then may be published on a public careers page or internal portal.
- **CandidateApplication:** records each applicant applying to a position – including personal info, contact, source (e.g. via career site or referral), current status in pipeline, etc., plus a link to their resume/CV (stored in DMS or as a file path).

- **Resume/CV Data:** Rather than storing large files in the DB, the CV file will be uploaded to the DMS (Document service) and associated with the application record. The DMS will return a file ID or URL that the recruitment module stores. This way, resumes are accessible but managed by the DMS (which can handle storage and virus scanning).
- **ScreeningResult:** stores results of automated CV screening for each application (e.g. extracted skills, experience years, a fit score or ranking). This can be a separate table or fields in CandidateApplication (e.g. `screening_score`, `rank`). It will be populated by an async process (see below).
- **InterviewSchedule:** to track scheduled interviews for candidates. This might include date/time, interview mode (online/offline), link to video meeting if online, assigned interviewer(s), and a reference to a set of interview questions.
- **InterviewFeedback:** records feedback and ratings from interviewers or automated video interview analysis. For example, if using an online video interview platform, the system can store scores or transcripts which can feed into candidate ranking.
- **Offer & Onboarding:** Once a candidate is selected, an Offer record can be generated (with details like offered role, salary, joining date). On candidate acceptance, the module triggers onboarding actions (like creating an employee profile).

Integration & Workflow: The recruitment process involves multiple integrations: - When a job is created, it could optionally integrate with an external job portal API or simply be made available on a **public careers page** (which would call an API like `GET /recruitment/jobs` to list open positions). Applications submitted via that page call an open endpoint (with CAPTCHA/security) to create a CandidateApplication. - **CV Screening:** Upon receiving an application with a resume, the service will enqueue a *"CV Screening"* job. A background worker (or an AI microservice) will pick up the job, parse the resume (using NLP/ML to extract skills, experience), and assign a score or shortlist recommendation. The Recruitment service can then update the application's record with these results and perhaps trigger an alert for recruiters for high-scoring candidates. This asynchronous design ensures the main application flow isn't delayed by heavy ML processing. - Based on screening results, the system can automatically advance or rank candidates. Recruiters can also manually shortlist candidates. The top candidates can be sent **automated interview invitation emails** – the recruitment module will call the Notification/Email service to send out templated interview invitations (with a scheduling link or fixed time). - For online interviews, the system might integrate with a video interview platform or support pre-recorded Q&A. In the latter case, a candidate could receive a link to record answers to preset questions. The backend could store those recordings in the DMS (or a specialized video store) and even use an AI service to evaluate responses (another possible async job). - After interviews, interviewers (who would have restricted access to the candidate data) can submit feedback via the HR portal, which the recruitment module saves. The module can then automatically **rank candidates** based on combined scores (CV and interview feedback) to assist decision-making. - Once a candidate is marked "Hired", integration with other modules kicks in: the **HR Core service** is notified to create a new Employee record (transferring relevant details from the candidate application), and the **IAM service** creates a user account with appropriate default roles (e.g. an "Employee" role in the tenant). These actions can be orchestrated by publishing an event like `candidate_hired` on the message bus, which the HR Core and IAM services subscribe to, decoupling the recruitment module from direct calls. Additionally, an **Onboarding workflow** could be triggered (e.g. using the Workflow service) – this might create a checklist of onboarding tasks in the Task module (IT setup, document submission, welcome orientation, etc., see Performance/Task module). - If the candidate was not selected, the system can send an automated rejection email (again via Notification service) and possibly archive the application or mark talent pool status.

APIs: Key endpoints would include: - CRUD for job positions (`/recruitment/jobs`), with only HR/recruiter roles authorized to create/modify. - Endpoints for candidates to apply (`/recruitment/jobs/`

`{jobId}/apply`) – likely an open or minimally protected endpoint, with input validation and perhaps rate-limiting (to prevent spam). - Endpoints for recruiters to manage applications: e.g. `/recruitment/applications?status=SHORTLISTED` to list, or actions like `POST /recruitment/applications/{id}/advance` (to move a candidate to next stage), `/.../scheduleInterview`, etc. - API to trigger offer creation and acceptance: `/recruitment/applications/{id}/offer` and `/recruitment/applications/{id}/hire`. - All these will enforce RBAC: e.g. only users with a Recruiter or HR role (and belonging to that tenant) can list or update applications. Candidates themselves might only access a limited endpoint to view their status if we provide such capability via a login, but typically external candidates won't log into the system beyond email communications.

Multitenancy & RBAC: All recruitment data carries a tenant ID, and the service will ensure queries and actions are limited to users' own tenant. A recruiter from Company A cannot see candidates or jobs from Company B. The RBAC model will likely introduce roles such as **Recruiter**, **Recruitment Manager**, and possibly **Interviewer**: - *Recruiters* can create jobs, view and manage all applications for their tenant. - *Interviewers* (other employees assigned to interview panels) might get limited access to view specific candidate info and submit feedback, but not see other jobs/candidates. - *HR Manager* role might by default have recruitment privileges as well, depending on policy. These roles and permissions would be managed in the IAM module (for example, a permission like `recruitment:manage` for full ATS access). Authorization checks will verify both role and tenant scope before allowing operations.

Scalability & Async Considerations: Recruitment can have bursts of activity (e.g. when many candidates apply to a popular job posting). The stateless service can be scaled horizontally behind the gateway to handle spikes. Expensive operations like document parsing or video analysis are done asynchronously so they can scale separately (e.g. a pool of ML workers). We will use the queue to distribute these tasks and ensure they are **idempotent** (so if a worker fails and retries, the outcome remains correct). The service will also log all major events (job created, candidate advanced, offer sent) to provide an audit trail. This is important for compliance and to track the hiring process, and can help debug issues (like "why did candidate X not get an email?").

Overall, the Recruitment/ATS module will seamlessly integrate hiring into the HRMS, automatically creating employee records on hiring and providing a smooth experience from application to onboarding – addressing the challenge of **manual recruitment consuming too much time** by automating notifications, tracking, and screening.

2. Employee Self-Service Portal (ESS)

Role & Scope: The Employee Self-Service Portal is primarily a front-end product (web or mobile app) that allows employees to access their own HR information and perform common actions. While it is not a separate "backend" module per se, it relies on a suite of **API endpoints provided by other services**. Our design ensures that all features needed by the ESS portal are exposed via secure APIs. Essentially, the portal is an aggregation of various backend capabilities presented to the end user (employees and managers). Key features from the requirements include: viewing leave balances, checking attendance/timesheets, downloading salary slips, accessing personal documents, viewing the org chart/reporting structure, etc. We ensure each of these is backed by an API.

Backend Support: For each self-service feature: - **Leave Balance:** The Leave service (see next section) will provide an endpoint like `GET /leave/balance` which returns the current user's leave entitlements and remaining balance for each leave type. When an authenticated employee uses ESS to view leave balance, the portal calls this API (with the employee's token which includes their user ID/tenant). The Leave service logic will fetch that employee's record and respond. - **Apply for Leave:**

Similarly, an ESS page for leave application will call `POST /leave/requests` with details of the leave. The Leave service will create the request and initiate approval workflow. The response (or subsequent notifications) is shown back to the user. - **Attendance & Timesheet:** The Attendance service will expose endpoints like `GET /attendance/timesheet?month=2026-02` to fetch the logged attendance for the current user for a given period. The ESS uses this to display check-in/out times, total hours, overtime, etc. If real-time clock-in/out is allowed via the portal, an endpoint `POST /attendance/checkin` can be used when the user clicks a "Clock In" button (with appropriate validations, like not allowing duplicate check-in). - **Download Salary Slips:** The Payroll service will store or generate pay slips (likely PDF files). An endpoint `GET /payroll/payslip/{month}` can be provided. When an employee requests a slip, this API either returns the PDF file (perhaps via the DMS if stored there) or triggers generation on the fly. For security, this must verify that the requesting user is indeed the owner of that payslip (or their manager/HR). We may use a secure mechanism like a one-time URL or direct download through the gateway after auth. - **Personal Documents:** The DMS will have endpoints to list and fetch documents associated with the logged-in user. For example, `GET /dms/documents?employeeId=ME` could return a list of the employee's own documents (CV, certificates, contracts, etc.), and `GET /dms/documents/{docId}/download` to retrieve a file. The ESS portal can then allow the employee to view or download their documents. Role checks ensure employees can only access their own documents (or those they are permitted to see). - **Reporting Structure:** The HR Core service will manage the organization structure (each employee likely has a manager ID, department, etc.). We can provide an endpoint like `GET /hr-core/org-chart` or `GET /hr-core/manager/{employeeId}/team` to get information on an employee's manager and team hierarchy. For ESS, typically an employee can see their immediate manager and possibly their reports if they are a manager. A manager using ESS might have an extra view to see their team's profiles or leave status. These queries are served by HR Core service (which contains employee profiles and their manager relationships). - **Profile Updates:** Though not explicitly listed, often ESS allows employees to update certain profile fields (address, emergency contact). If needed, the HR Core can expose a limited update endpoint for self-service that only allows certain fields and perhaps requires approval (tracked via workflow).

API Gateway & BFF: The ESS portal will consume many microservice endpoints. We have two options: 1. The front-end calls each service API (through the gateway) individually to assemble the dashboard (e.g. call leave balance API, then attendance API, etc.). This is feasible because the gateway can route to multiple services under one domain. However, it can result in multiple calls on page load. 2. Implement a **Backend-For-Frontend (BFF)** pattern: create a lightweight aggregation service or endpoint that calls multiple internal APIs and composes the data. For example, a `GET /self-service/dashboard` endpoint could internally call leave, attendance, payroll services to gather an overview for the logged-in user. This approach reduces the number of round trips and encapsulates logic for the front-end, at the cost of an extra layer.

Given our architecture, we can initially let the front-end directly use the individual service endpoints via the gateway (since they are all authenticated consistently). The API Gateway can simplify this by offering a unified authentication and routing – the user's JWT token (issued by IAM) is sent with each request and the gateway forwards it to services. Each service will verify the token and authorize actions, possibly consulting IAM for roles if needed.

Security: The ESS is strictly for authenticated users. We must ensure that an employee can only access their own data (or data of their subordinates where applicable). This is enforced at the service layer with RBAC and additional checks: - For example, even if an employee tries to call another employee's payslip endpoint, the Payroll service will check the token's user ID against the requested employee ID and deny if they don't match or the caller isn't an authorized manager/HR. - We will leverage the **roles and hierarchy** in IAM/HR Core. A manager's token might have a claim or can be looked up to allow certain team data. Otherwise, default employees only see self data. The WorkOS guidance on multi-tenant

RBAC is applicable: roles are tenant-scoped and no cross-tenant data is ever returned [7]. In practice, the tenant ID in the token must match the data's tenant and the role must permit the action.

UX Consideration: The self-service portal essentially unifies the user experience. From a backend perspective, our design supports it by maintaining consistency across modules. Using a single sign-on (SSO) concept, once the user logs into the HRMS (handled by IAM service), they can access all authorized services without separate logins – the token is trusted system-wide. (In the Indium case study, they explicitly integrated ESS and other modules with SSO [9], which we achieve via the unified IAM service).

Scalability: The ESS usage will be high (all employees). Because it is just calling backend services, the scaling mainly happens in those services. We ensure the gateway and services can handle concurrent requests (e.g. by horizontal scaling). Caching might be applied for some data that doesn't change often – e.g. organization structure or leave policy info – to reduce load on the database for frequent ESS queries.

In summary, we are not building a separate "ESS service" but rather **exposing a comprehensive set of APIs** that the self-service front-end will use. The design emphasizes clear, role-secured endpoints for every piece of functionality. This gives employees transparent access to their information (addressing transparency and time-saving goals) without compromising security.

3. Attendance Tracking Service

Purpose: The Attendance service automates the tracking of employee work hours, attendance, and overtime – addressing the difficulty of tracking attendance manually. It will record daily logins/logouts (or shifts), calculate working hours, flag anomalies (late, missing entry), and produce reports. It works closely with Leave management (to account for absences) and Payroll (to provide hours data for salary calculation).

Data Model: In a new `attendance` schema, core tables include: - **AttendanceRecord (Daily):** Each record represents an employee's attendance for a day (per tenant). Key fields: `employee_id`, `date`, `check_in_time`, `check_out_time`, and derived metrics like `total_hours` or minutes worked. It might also have flags (e.g. `absent` or `on_leave` if no check-in but on approved leave) and an `anomaly` field if something is irregular (like missed punch-out). Alternatively, we might store raw punch events (multiple per day) and compute totals via logic, but a daily aggregate table simplifies payroll integration. - **AttendanceEvent (Raw logs):** If needed, this table logs each punch event or entry (timestamp, type in/out, perhaps device or IP if from different sources). This can help with audit and in/out tracking if people sign in/out multiple times. - **OvertimeRecord:** Could be separate or included in daily. If overtime needs special tracking (hours beyond standard work hours), we might calculate and store them either in the daily record or a monthly summary per employee. Overtime rules (e.g. counting after 8 hours/day or 40 hours/week) can be applied and extra hours stored here. - **Shift/Schedule:** Optionally, a table defining expected working hours or shift timings per employee or role, to compare actual hours against scheduled. This is more advanced (e.g. if different employees have different shifts). - **Holiday/OfficeClosures:** A reference table (tenant-specific) listing official holidays, to ignore those in attendance expectations.

The attendance service uses the above to determine if an employee was present, late, or absent on a given day.

Data Capture: There are multiple ways attendance data might enter the system: - **Manual/Web Clock-in:** Employees can log attendance via the ESS portal or an attendance UI – clicking "Clock In" and "Clock

Out". These actions will call endpoints (`POST /attendance/check-in` or `/check-out`), which create AttendanceEvent entries and update the current day's AttendanceRecord for that user. - **Biometric Device Integration:** Many enterprises use biometric scanners or RFID swipes. Our system can integrate by providing an API for device software to call, or by periodically importing logs. For example, a small service or script could push data to `/attendance/events` endpoint with employee ID and timestamp when a device logs an entry. The Attendance service could also poll a device's database if direct push isn't available. This design allows hardware integration for automation. - **Advanced (Camera/AI):** In some cases, companies use AI-based attendance (face recognition cameras). The data model could accommodate this (e.g. storing recognized face events). In fact, the user's provided **Attendance redesign** diagram suggests an advanced approach with tables like "employee_faces", "cameras", "videos", etc., implying a vision-based attendance tracking system. Our architecture can support this as an extension: a specialized sub-component could process video streams, detect faces, match to employee records, and then feed the Attendance service with "check-in" events when a face is recognized at a door【16†】. While implementing that is complex, our modular design ensures we could add such a component without altering other modules – illustrating continuous innovation capacity.

Business Logic: - **Automatic Calculations:** When an employee checks out (or at day's end via a scheduled job), the service computes `total_hours` = checkout – checkin minus breaks (if recorded). Overtime can be calculated if hours exceed the standard work day (configurable per tenant, e.g. >8 hours = overtime). This can be stored in an overtime field. - **Leave Integration:** The Attendance service should consider approved leaves. If an employee has a full-day leave approved on a date, either (a) no attendance record is created for that day and payroll will treat it as paid leave via the Leave data, or (b) an attendance record is created marked "On Leave" for completeness. We will likely rely on the Leave service's data rather than duplicating it. A simple approach: a daily scheduled job can cross-check any day where an employee has no check-in – if they have an approved leave for that day (via Leave service API), mark the attendance record as "leave" (not absence). If neither attendance nor leave, then it's an **absent** day which may trigger alerts. - **Adjustments:** The system allows **HR to adjust records** if needed. There might be an endpoint for HR/Attendance admins like `PUT /attendance/{employeeId}/records/{date}` to correct a missing time or adjust hours (with proper audit logging). This addresses scenarios like someone forgot to clock out and HR needs to fix it.

Notifications & Reports: - The service can send alerts for anomalies. For example, if someone didn't clock in by a certain hour and no leave is logged, it could notify their manager or HR. These could be implemented with scheduled checks and the Notification service for sending emails. - Monthly attendance reports can be generated by this service. An endpoint `GET /attendance/report?month=X` for HR might produce a summary of all employees' present/absent/overtime stats. These could be generated on-demand or pre-computed at month-end. We might offload generation of heavy reports to a background job (especially if generating a PDF or Excel).

APIs: - **Employee-facing:** `POST /attendance/check-in` and `.../check-out` (or a combined endpoint that toggles status) – these record timestamps. `GET /attendance/my-records?month=Feb` – returns the logged times for the current user for that period. - **Manager/HR-facing:** `GET /attendance/records?date=2026-02-05&team=XYZ` – for a manager to see today's attendance for their team, or for HR to see for entire tenant. `GET /attendance/{employeeId}/records?month=...` to inspect a specific employee (restricted to those who have permission). - **Adjustments:** `PUT /attendance/records/{recordId}` for HR to edit or `POST /attendance/records` to create a missed record. - **Device integration:** perhaps a secure endpoint or a separate ingestion mechanism that can accept data from time-clock systems (this might be outside user-facing API, more of an internal or partner API with an API key).

Multitenancy & RBAC: All records include tenant_id and the service will enforce that only users from that tenant can access them. Regular employees can only access their own attendance data (the service will match `employeeId` from the token to requested data). Managers might have a role or a specific claim allowing them to view their subordinates – the service can check with HR Core to ensure the target employee's manager_id matches the requester. HR or Admin roles can view all for the tenant. These rules are implemented via IAM roles (e.g. a "Manager" role and logic for subordinates) and checks within endpoints.

Scalability: Attendance events are frequent (every employee clocks in/out daily). This module will generate a high volume of writes. However, the data per entry is small and straightforward. We will optimize by using batch inserts (if a device sends bulk data) and ensure proper indexing on (tenant_id, employee_id, date) for queries. The service can be scaled horizontally. If needed, we can partition attendance data by month or year to keep tables manageable for very large organizations. In a multi-tenant DB, this isn't usually an issue unless one tenant is extremely large, in which case sharding by tenant or moving that tenant to a separate DB could be considered later [10].

We will also implement **retention policies**: e.g. raw events might be archived or purged after a year if not needed, to keep DB lean (the daily aggregates are enough for long-term).

Fault Tolerance: Since attendance is crucial, we ensure that even if the service goes down briefly, employees can still register attendance (perhaps by having an offline mode on devices to cache and send later). The system should recover gracefully – e.g. if the queue of events builds up, it processes them in order once back.

In summary, the Attendance module provides a robust back-end for capturing time logs and computing hours automatically, feeding reliable data into payroll and minimizing manual tracking errors. It aligns with the goal of **attendance automation** and transparency for both employees and managers.

4. Leave Management Service

Purpose: The Leave service manages employee leave (time-off) requests and balances. It replaces manual or email-based leave processes with an online system for applying, approving, and tracking leaves. It also automates notifications and updates leave quotas. This addresses the challenge of tracking leaves and multi-level approvals manually.

Data Model: In the `leave` schema, key tables: - **LeaveRequest:** representing a single leave application. Fields: `id, employee_id, tenant_id, leave_type, start_date, end_date, reason, status (Pending/Approved/Rejected), requested_at`. It may also store `approver_id` or next approver in case of multi-level. - **LeaveBalance / Entitlement:** tracks how many days of each leave type an employee has per year (or current balance). For example, 20 annual days per year, 5 sick days, etc., and how many remain. This table (`employee_id, leave_type, year, allocated, used, carryover`) allows the system to calculate balances. It's updated when requests are approved. - **LeaveType/Policy:** a reference table defining leave types (Vacation, Sick, etc.) and rules per tenant. This might include annual allocation, max carryover, whether partial days allowed, approval requirements (e.g. some might not need approval like optional holidays). HR can configure this for each tenant's policy. - **LeaveApproval Workflow:** We might not need a separate table if we integrate with the generic Workflow service. However, we could have a table for approval steps if implementing ourselves (like `leave_request_approvers` with `request_id, level, approver_role/ID, status`). If using the Workflow engine, it will track approval progression outside of the Leave schema, and the LeaveRequest.status will simply be updated to Approved/Rejected at the end.

Process Flow: - **Request Submission:** An employee initiates a leave request via ESS (or HR portal for someone else). The Leave service receives a `POST /leave/requests` with details. It validates (e.g. ensure start <= end, they have enough balance if required, no conflicting dates with prior approved leave or other absences). The request is saved with status "Pending". - **Multi-Level Approval:** According to requirements, leave might require manager approval, then HR approval. This can be handled by the **Workflow service** to keep it configurable. When a request is created, the Leave service calls the Workflow service to start a new workflow instance (e.g. a "LeaveApprovalWorkflow" template defined for that tenant or globally). The workflow engine knows the sequence: send to employee's manager first, then upon manager approval, to HR. The Workflow service will communicate back the decisions (it could call back via webhooks or we simply poll it). Alternatively, the Leave service itself can implement a fixed chain: find the employee's immediate manager (via HR Core), assign the request to them; when they approve, assign to HR role. - A simple approach is to store `approver_id` in LeaveRequest, initially set to manager. An email notification (via Notification service) is sent to the manager. The manager uses their portal to approve (`POST /leave/requests/{id}/approve` – allowed only if they are the current approver). The service then moves the request forward: if a second level is needed, it changes `approver_id` to an HR role user (or marks it awaiting HR) and sends notification to HR. Finally, HR approves and the request status becomes Approved. This approach works but hardcodes two levels. - The Workflow service approach is cleaner: it can manage arbitrary levels. For our design, we plan to utilize the Workflow module (since it exists) by registering a workflow definition for leave approvals. The Leave service would then just initiate and wait for outcome. - **Notifications:** At each stage, the system sends emails or in-app notifications. After final approval or rejection, it **auto-notifies the employee** (requirement: "Auto Email Notification after approval/rejection") and possibly their manager. The Notification service is used for all these emails. The content can be template-driven (like an email template that fills in employee name, dates, status, etc.). - **Balance Update:** When a leave is approved, the LeaveBalance for that employee and leave type is decremented by the number of days taken. This happens immediately upon final approval. If a request is rejected or canceled, no balance change. If a request is partially approved (say, dates adjusted), the system should handle that, but perhaps out of scope unless we implement editing. - **Integration with Attendance:** On approval, the service should inform the Attendance module or mark those dates so they are not counted as absences. This could be done by an event or direct call. For example, for each approved leave, the Leave service might emit an `employee_on_leave` event with employee and dates, which the Attendance service can consume to mark those days appropriately (or simply the payroll will check leave data to avoid deducting salary). - **Edge cases:** HR should be able to **adjust leave balances** (requirement mentions HR can adjust leave balances manually). We can support that via an admin endpoint or a UI: e.g. HR enters an adjustment (increase or decrease in days for an employee's entitlement) – the Leave service then updates the LeaveBalance table and logs the change (with reason). This might be needed for corrections or special cases (like bonus leave or deducting leave without pay).

APIs: - Employee: `POST /leave/requests` (apply for leave), `GET /leave/requests?mine=1` (view their submitted requests and statuses), possibly `DELETE /leave/requests/{id}` (to cancel a pending request). - Manager/HR: `GET /leave/requests?team=1` (manager sees requests of their team awaiting approval), `POST /leave/requests/{id}/approve` or `/reject` (to action a request). HR can have a similar endpoint or use the same if they are assigned as approver in second level. - HR Admin: `GET /leave/balances?employee={id}` (view/adjust an employee's leave balances), `PUT /leave/balances/{employeeId}/{leaveType}` to adjust. - Reference: `GET /leave/policies` (to retrieve leave type definitions, maybe for UI to show how many days allowed per type).

All these endpoints enforce RBAC: employees can only create for themselves (or possibly their direct reports if acting on behalf? Usually not needed except HR who can record leave on behalf of someone).

Managers' approve endpoint checks the user is indeed the pending approver for that request. HR's endpoints are restricted to HR roles.

Multitenancy & Configurability: Leave policies differ by tenant (some give 15 vacation days, some 30, etc). Our LeaveType/Policy table is keyed by tenant. We'll allow each tenant's HR to configure basic rules (perhaps via an admin UI that updates this table). The Leave service logic will reference these rules (e.g. if an employee requests more days than available, it will reject or mark as unpaid if policy allows going negative). This flexibility ensures the system can be customized per tenant without code changes [11].

Cron/Scheduled tasks: - **Accruals:** Some companies accrue leave monthly (e.g. 1.5 days earned per month). We can schedule a monthly job that increments each employee's LeaveBalance according to policy. Alternatively, the system can credit the full annual allocation upfront and just let it decrement (simpler). For advanced needs, an accrual schedule can be implemented. - **Carryover & Reset:** At the end of the leave year (often Dec 31 or employee work anniversary), a job should run to carry over permitted unused leave to next year and reset current year's counts, as per policy. This can be a cron job that runs yearly and updates balances (storing maybe last year's carryover in a separate field). - **Reminders:** A periodic task can remind managers of pending approvals (e.g. daily email if a request has been pending > X days) to prompt timely action. - **Leave Calendar:** Not a job, but the service could offer a calendar feed or integration so employees can see who is on leave (maybe by team). The data is already there, so the front-end can display it; no separate backend action needed aside from queries.

Scalability: Leave management workload is moderate – occasional requests and updates, not as constant as attendance. The service can easily scale horizontally. The main heavy lifting might be computing reports (like who took how many days – but those are small queries if indexed by date). The multi-level approval adds some complexity, but our use of the Workflow service or careful logic prevents any single request from becoming too slow (each approval is a quick action). We will maintain consistency by possibly using transactions when updating balances and request status together, to avoid race conditions (though race is low probability in leave context).

Reliability: The combination of synchronous and asynchronous steps (requests vs. email sending) means we should ensure no part fails silently. For example, if an approval email fails to send, the system should retry via the Notification service queue or at least log it so HR can see and manually inform if needed. Using the Notification service decouples that – the Leave service just posts a notification request and doesn't block on email success.

In conclusion, the Leave Management service provides an end-to-end automated solution for leave applications: employees apply online, approvals are routed systematically, balances update automatically, and notifications keep everyone informed. This meets the need for **multi-level approval with automatic notifications and balance tracking**, eliminating errors and delays from manual handling.

5. Payroll & Compliance Service

Purpose: The Payroll service handles salary calculations, pay slip generation, and compliance with tax and deduction rules. It transforms attendance and leave data into monthly salary outcomes, automates pay processing, and ensures deductions (tax, provident fund, etc.) are correctly applied. This addresses challenges like payroll calculation errors and manual effort by providing an automated, rules-driven payroll process.

Data Model: A dedicated `payroll` schema will maintain: - **EmployeeSalary (Salary Structure):** For each employee, store their current salary components and details. This could include basic salary, various allowances, deductions, tax preferences, etc. It might reference a **SalaryGrade** or template if many employees share structures. Also include effective date (to handle increments) and maybe a history of changes. - **PayRun:** Represents a payroll run for a given period (e.g. month) and tenant. Fields: `payrun_id, tenant_id, period (e.g. 2026-02), status (Pending/Processing/Completed), run_date`. When payroll is processed, a PayRun record is created. - **PaySlip/PayRecord:** Detailed results for each employee in a PayRun. Fields: `employee_id, payrun_id, gross_pay, net_pay, tax_deducted, provident_fund, etc`, plus maybe JSON or a separate table for line-item breakdown (basic, allowance, overtime pay, leave deductions, etc.). Essentially, this is the computed outcome that will be presented on the salary slip. - **DeductionRules & TaxTables:** Reference tables for compliance logic. For example, a table of income tax slabs per country or state, percentage rates for provident fund or social security, professional tax amounts per salary range, etc. These can be pre-populated and updatable as laws change. Tenant-specific override might be allowed if needed (most of these are government mandated so probably global per region). - **IncrementSchedule:** Table to track planned salary increments (as part of "Increment Cycle Management"). It could have `employee_id, effective_date, new_salary_amount or percentage increase, approved_by` etc., to schedule future salary changes. The payroll process can reference this to apply changes at the right time. - **Loan/Advance:** If handling advance salary or loans, a table for advances given (`employee_id, date, amount, to_be_recovered_by` etc.), so payroll can deduct installments or adjust net pay accordingly.

Payroll Calculation Workflow: 1. **Payroll Configuration:** First, HR defines the payroll frequency (monthly, biweekly, etc.) and payday for each tenant. Assume monthly for simplicity with payday say end of month or a specific date. 2. **Attendance/Leave Data Gathering:** When it's time to run payroll for a period, the Payroll service will fetch needed data: - Total working days in the period, days present for each employee, days of leave (paid or unpaid), overtime hours, etc. This can be done by querying the Attendance and Leave services. For example, it might call `GET /attendance/report?month=Feb&employeeId=all` and `GET /leave/used?month=Feb` or similar, or use events if those services publish summary events. If cross-service calls are heavy, this can be delegated to a background job. - Alternatively, maintain a small cache or summary table updated daily; however, real-time queries are fine if optimized. 3. **Calculate Gross & Net:** For each employee: - Start with fixed gross salary from EmployeeSalary. - Add any variable components (e.g. bonuses, which HR could input into a "Bonus" table each cycle or an ad-hoc entry). - Deduct for unpaid leave or absences: If an employee took leave without pay days, compute deduction = (monthly salary / working days) * unpaid days. Our integration ensures we know how many unpaid days (approved leave that is unpaid, or absences). - Add overtime pay if applicable: e.g. if overtime hours recorded, apply overtime rate (maybe 1.5x hourly rate). - The result is adjusted gross for that month. - Apply **taxes and deductions**: use the rules tables to calculate income tax withholding (TDS). For example, if the employee's annual salary falls in certain slab, compute monthly tax to deduct. Also deduct fixed contributions like provident fund (e.g. 12% of basic) and professional tax (flat amount based on state and salary). - Deduct any loan/advance installments: if an advance salary was given and scheduled to recover over N months, subtract that portion. - Finally compute net pay = gross – total deductions. - This calculation can be done via code, but to be enterprise-ready, making it **configurable** is ideal. We could implement a rule engine or at least make tax rates editable via the admin UI. For example, store formulas or use a small scripting language or decision table for calculations. Initially, hardcode typical rules but allow overrides as needed. 4. **Generate Pay Slips:** For each computed record, format a pay slip. This could be done by filling an HTML/PDF template with the line items and amounts for that employee. The slip should include company info (maybe from Tenancy schema), employee info, pay components, and statutory info (tax PAN, etc.). - The PDF generation can be asynchronous if many slips (each slip generation job put on the queue, workers create PDFs). Or generate on-demand when the user requests it. To reduce load at

peak, we might generate all slips once and store them in DMS with a reference in PayRecord. Storing is convenient for later retrieval and compliance archiving. - In any case, employees can access via ESS as discussed. 5. **Post-Payroll Actions:** Once payroll is computed, the service marks the PayRun as completed and can trigger: - **Notifications:** email employees that salary slips are ready (maybe with a link or attachment for convenience). Or simply notify that salary has been credited (actual bank transfer might be outside system scope). - **Bank transfer file:** If needed, the system can generate a bank transfer advice or integration file listing all net pays to be transferred. Many payroll systems output an "ACH file" or similar which the finance team uploads to the bank. We could incorporate that if desired by generating a CSV with account numbers and amounts. Account details (bank account, IFSC) would be stored in HR Core for each employee. - **Compliance reports:** e.g. monthly tax report, PF report, that the company needs to remit. These can be generated similarly to payslips using the aggregated data. Possibly, add an endpoint to get totals of tax deducted etc., or even automate filing via an API if available (that's beyond our scope for now, likely just generate reports).

Increment Cycles: For managing increments (salary raises, typically yearly or performance-based): - The HR or managers can plan increments and input them (perhaps in the IncrementSchedule table). When the effective date arrives, the Payroll service (or HR Core) should update the EmployeeSalary structure. This can be done by a scheduled job checking for any increments effective this month and applying them. It might also trigger generation of an **Increment Letter** (similar to an offer letter, stating the new salary). The Document/Notification service can be used to deliver this letter to the employee on their increment date. - The payroll calculation should be aware of mid-period changes: if increment effective mid-month, either proration is done or effective next period as per company policy. We can simplify by assuming increments align with period boundaries.

Advance Salary: If an employee requests advance salary (or a loan), HR can record it in the system. The Payroll service will then either: - Pay the advance immediately (if integrated with payment or just record it and let HR actually give money offline), and schedule deductions in future payrolls. - Or, if "advance salary" means paying earlier than payday, that complicates the cycle. Likely, it's more about loans/advances to deduct from future salaries. We handle that as described with a loan table and deduction logic.

APIs: - **Payroll Run Control:** `POST /payroll/run?period=2026-02` – initiate payroll calculation for a given period. This could be restricted to HR/Payroll roles. It might kick off an async process and return a run ID. The actual heavy lifting can be done by background workers picking up this run ID from a queue (to avoid blocking the request). - **Payroll Status/Report:** `GET /payroll/run/{runId}` – check status (Processing or Completed) and summary. `GET /payroll/run/{runId}/results` or `/payslips?employeeId=...` – to fetch the detailed results. HR could use this to verify or adjust. - **Payslip Access:** `GET /payroll/payslips/{period}` for the current user (employee) to get their slip (PDF or data). - **Salary Management:** `GET /payroll/salary-structure/{employeeId}`, `PUT /payroll/salary-structure/{employeeId}` (for HR to update salary, e.g. when giving an increment or correcting a component). Possibly `POST /payroll/increment` to schedule a batch increment (some systems allow uploading increment info for many employees – but we can also do one by one). - **Loans/Advances:** `POST /payroll/advance` to record an advance given, etc., (HR role). - **Reports:** `GET /payroll/compliance/tax?period=Q4` – produce a tax deduction summary; `GET /payroll/compliance/pf?month=X` – produce PF contributions list, etc.

RBAC & Security: Payroll data is highly sensitive. Only authorized roles (Payroll admin, HR) should be able to run payroll or view other employees' salary info. Regular managers typically should **not** see their subordinates' salaries in most companies (except maybe if explicitly allowed for some). So by default, restrict salary details to HR/Payroll roles. Employees of course see their own payslips. We will enforce

this via IAM roles and service checks. E.g., an employee token requesting another's payslip gets denied. The HR or Payroll officer role will have privileges to view and edit salary data across the tenant. Actions like initiating a pay run or adjusting salary could be limited to a *"Payroll Admin"* role, which only a subset of HR users have.

We also consider **data protection**: we might encrypt certain fields at rest (like bank account numbers, or salary amounts) in the database for compliance with security policies. The application can decrypt (with keys managed securely) when needed for processing, but if someone directly accesses the DB, they wouldn't see plain values. Additionally, audit logging is critical – any change to salary data or any payroll run should be logged with who did it and when, for accountability.

Scalability & Performance: Payroll processing can be heavy but occurs infrequently (e.g. monthly). We have designed it to run as background jobs so it doesn't time out user requests. If the tenant has thousands of employees, the computation and PDF generation may take minutes – which is acceptable as a batch job. We ensure the job queue can distribute the load (maybe process 100 employees per worker in parallel). The database will see a burst of reads (fetching attendance/leaves) and writes (inserting pay records) during payroll run – indexing and perhaps caching of attendance data can help. Using read replicas could be considered: e.g. heavy reads from attendance and leave services could go to replicas to not slow the primary. However, given the moderate size of typical tenants, this might not be necessary initially.

Compliance: The service is built to adapt to compliance needs: - When tax rules change, updating the rules table will alter calculations. - If a new deduction or contribution is introduced (say a new social security type), we can add a field or table and adjust the formula without redesigning the entire system (thanks to modular structure). - We generate necessary outputs for compliance (like forms or reports), which can be added to over time.

By automating payroll with this design, we ensure that **attendance-based salary calculation, bonuses, tax deductions, professional tax, etc.** are all handled in one integrated workflow. This significantly **reduces errors** compared to manual calculations and ensures employees are paid correctly and on time. It also provides transparency (employees can see their payslip details and how each component was computed), fulfilling one of the business benefit goals.

6. Document Management & Exit Management

These two are grouped because exit management will heavily involve document generation and tracking, though they are distinct features. We will discuss each in turn, noting their interactions.

Document Management (DMS) Enhancements

The HRMS already includes a Document Management System (`dms` schema) which likely stores employee-related documents and templates. We will augment this to cover the new requirements like document expiry alerts and broader document access.

Scope: The DMS stores files such as resumes, certificates, contracts, policy documents, letters, etc., along with metadata. Key features to implement: - **Expiry Alerts:** Many documents (identity proofs, certifications, visas, contracts) have expiration dates. We will add an `expiry_date` field to the document metadata. HR (or the employee) can set this when uploading a document (e.g. passport copy expires on X date). The system will then automatically remind relevant parties before expiry. - **Document Categorization:** Introduce a document type or category field (e.g. "ID Proof", "Offer Letter", "Experience Letter", "Policy Document") to help classify and apply rules (like which ones need expiry

tracking). - **Access Control:** Ensure that sensitive documents are only accessible to authorized roles. For example, an employee's performance review document might only be visible to HR and that employee's manager, not to the employee themselves (depending on policy). We can manage this via a permissions table in DMS linking documents to roles or via naming conventions and logic. However, a simpler approach is to assume: - Personal documents (uploaded in employee's profile, like their certificates) can be seen by that employee and HR. - Company documents (policies, templates) visible to all employees or relevant roles. - We will implement checks accordingly in DMS download API calls. - **Integration with other modules:** Recruitment will store CVs here; Payroll might store payslips or tax forms here; Exit will generate letters to store here. So DMS should provide an easy way for other services to save and retrieve documents. Likely, a service will call DMS API with a file or file payload, and DMS returns an ID/URL. That ID can be stored in the calling service's record. The actual file is saved to disk or cloud storage with the metadata in the DB. - **File Storage:** Use a scalable file store. For example, use AWS S3 or Azure Blob for actual file content and just store keys in the DMS DB. This allows virtually unlimited storage and reduces DB load. If cloud not an option, at least use a network file system.

Notifications (Expiry): A scheduled job (daily or weekly) will scan the DMS for any documents nearing expiry (say 30 days out). For each, it will send a notification: - To the employee (if it's their personal document like passport) to remind them to renew. - To HR as well, if HR needs to ensure compliance (e.g. work permit). This can leverage the Notification service to send emails, or create a task in the Task module for the employee ("Renew your document X by date Y").

APIs: - `POST /dms/upload` – for uploading a new document. This can accept file binary or perhaps require a pre-signed URL approach for large files (to avoid gateway handling huge files). But simplicity: small files can be passed directly. Metadata like type, employeeId, expiry_date, etc., are sent along. Only authorized users can upload (e.g. an employee uploading their own doc, or HR uploading on behalf). - `GET /dms/documents/{id}/metadata` and `/download` – to fetch info or download a document file. The service will verify the requester's permissions. - `GET /dms/documents?employeeId=X&type=Y` – search/list docs for an employee or of certain type (for UI listing). - Possibly `DELETE /dms/documents/{id}` – if needed, though typically we might not allow deletion of important records except by admin. - The internal API for other services might be similar to upload but with a different authentication (services could have a service account or token to call DMS directly, or go through on behalf of a user).

RBAC: The DMS will use the user's roles and the document's metadata to enforce access. For instance, if a document is marked as "HR Only" in metadata, then only HR role can download. If it's an employee's personal doc, that employee and HR can. We may implement a simple rule: by default, documents linked to an employee (with employee_id field) are accessible to that employee and HR of that tenant; documents without employee_id (global, like policies) are accessible to all in tenant or specific roles as flagged. For more fine-grained control, a mapping table (document_id -> role that can access) could be used, but might be overkill initially.

Scalability: Storing and serving files can be heavy. Offloading to cloud storage means the service might simply generate pre-authenticated URLs for clients to download files directly from storage, reducing load on our service. If not, the service can stream files. We design it with streaming to handle large files. The DB queries for metadata are light and can be indexed by tenant, employee, etc. We should also virus-scan files on upload (enterprise best practice) – we might integrate an antivirus scanner service or use cloud storage that has scanning. This ensures malicious files don't end up stored.

By enhancing the DMS, we ensure **documents and compliance tracking issues** (original business challenge) are addressed – all documents are centrally stored, easy to retrieve, and the system actively alerts for renewals or expiries so nothing falls through the cracks.

Exit (Offboarding) Management

Scope: The Exit Management module oversees the process when an employee leaves the organization, whether through resignation, retirement, or termination. It ensures a standardized process including resignation submission, approvals, notice period tracking, knowledge transfer, asset return, and issuing of experience letters or other exit docs. This addresses the offboarding part of the HR lifecycle, ensuring nothing is missed when an employee exits.

Data Model: Possibly a new schema `offboarding` or integrate in `hr_core`. It might include: - **ResignationRequest:** If employees can initiate resignation in the system. Fields: `id, employee_id, notice_date (the date of submission), last_working_day (expected), reason, status (Pending, Accepted, Rejected), approver_id`. This is similar to LeaveRequest in flow. If the company allows digital resignation, employees use this. - **TerminationRecord:** HR might initiate separation (involuntary) – could reuse the same table with a flag or have separate type. But for simplicity, one table for any exit event. - **ExitChecklist:** A set of tasks that must be completed when someone is exiting. This could be represented by linking to the Task/Performance module (auto-create tasks for exit), but we might also store a checklist status here. For example, items like "Manager approved resignation", "Knowledge transfer completed", "Assets returned", "Exit interview done", "Experience letter given" – each as a column or a related table of checklist items. - **AssetAssignment:** If not already in system, a table listing company assets assigned to employees (laptop, badge, etc.). This would be updated at exit – ensuring each asset is returned. If an asset management system exists, we'd integrate; if not, a simple list here would do with fields: `asset_id, asset_type, assigned_to, status (assigned/returned)` – at exit, mark returned. - **ExperienceLetterTemplate:** Possibly store a template (could be in DMS) for experience letters, to generate a personalized one for the employee. Or store the generated letter in DMS per employee.

Process Flow: - **Resignation Submission:** An employee logs into ESS and submits a resignation request (if that's the policy) through an endpoint like `POST /exit/resignations` with desired last day and reason. The system creates a ResignationRequest with status Pending and routes to the manager (or HR directly, depending on policy). This triggers an approval workflow akin to leaves: typically manager approves resignation and then HR gives final approval/acknowledgement. - **Approval Workflow:** Use Workflow service or built-in logic: manager receives a notification, approves or discusses. If manager approves, HR is notified to finalize. If either rejects, the employee is informed (maybe they withdraw or discuss). - **Notice Period Tracking:** Once accepted, the clock starts on the notice period. The system knows the last working day (either provided by employee or calculated as submission date + notice period from company policy). It should track how many days left. Possibly send reminders a week before last day to both employee and HR about pending items. - **Knowledge Transfer & Tasks:** Upon approval of a resignation, the system **auto-generates tasks** in the Task/Performance module to ensure knowledge transfer: - For the exiting employee: tasks like "Document your current project status", "Complete knowledge transfer to person X", etc. - For the manager or replacement: tasks like "Obtain documentation from departing employee", "Assign someone to take over responsibilities". - These tasks ensure that critical information isn't lost. The Task module will monitor their completion. The exit module can query task status or simply rely on HR to confirm before final clearance. - **Asset Return:** If we have AssetAssignment records, the system generates tasks for the IT team or admin to collect items (e.g. laptop, access card). Alternatively, list them in an Exit Checklist UI for HR to mark off. The asset records should be updated to "returned" when done. - We could integrate with the Task system by creating a task for IT: "Collect laptop from John Doe" and one for Admin: "Deactivate access card of John

Doe". - **Exit Interview:** Often HR will conduct an exit interview. This can be scheduled outside system, but we can create a task "HR Exit Interview" as a reminder and mark it done when completed (notes from the interview might be stored manually or in a small field). - **Experience/Relieving Letter:** On the last working day (or after all tasks done), HR will generate an experience letter. Our Document service can store a template with placeholders (employee name, tenure dates, role). The Exit module (or HR through the interface) triggers generation of this letter, which is then saved to DMS and optionally emailed to the employee's personal email. There can be a `POST /exit/{employeeId}/generate-letter` endpoint for HR which returns the doc or stores it. This letter confirms their employment period and role, fulfilling requirement. - **System Access Removal:** As part of final steps, IAM should deactivate the user's account on the last day. This can be automated: e.g. a scheduled job on the day after last working day sets the IAM user status to inactive (and if integrated with SSO, possibly disables their login). It's crucial for security. We can implement this by publishing an event "employee_departed" on that day which IAM service listens to and then disables user, or simply direct call from exit processing logic to IAM to disable. - **Final Audit:** HR can have a checklist view to ensure all tasks are done, all assets returned, letter given, and user deactivated. Then mark the offboarding complete.

APIs: - Employee: `POST /exit/resignation` (if self-service resignation allowed), `GET /exit/status` (to see their request status and perhaps checklist items). - Manager/HR: `GET /exit/requests?team=...` (manager sees resignations of their team pending), `POST /exit/requests/{id}/approve` or `/reject` (manager or HR action). - HR: `GET /exit/checklist/{employeeId}` – to view that employee's offboarding checklist progress. `POST /exit/complete/{employeeId}` – mark offboarding complete (if all done). - For assets: if integrated, maybe `PUT /exit/asset-return/{assetId}` to mark returned, or do it via tasks completion. - `POST /exit/{employeeId}/letter` – to generate/download the experience letter.

Data Flow and Integration: The exit module heavily interacts with others: - **IAM**: to disable user access. Perhaps an integration event as described. - **HR Core**: update employee status (e.g. mark them as inactive or in an "exiting" state during notice). Also fetch manager info for approvals. - **Task/Performance**: create tasks for knowledge transfer and asset return. We will leverage the Task service (coming up) to actually handle the assignment and tracking of those tasks. - **DMS**: store the generated letters, and perhaps store any signed resignation letter if they upload one. - **Notification**: inform relevant parties of the resignation and send notifications (like a farewell note to team or just necessary alerts to IT etc., which can also be tasks).

RBAC: Obviously, an employee can only initiate for themselves. Managers can only approve for their direct reports. HR can see all resignations in their tenant. We also must restrict who can generate an experience letter – likely HR only. The asset management tasks would be visible to roles like IT or office admin as assigned. The system ensures no one outside the tenant can see these processes (tenant isolation).

Scalability and Reliability: Exits are relatively infrequent events. The load is low. The complexity is in coordination. We'll use the existing Workflow or Task systems to manage concurrency (e.g. tasks can be updated in parallel by different people). Consistency: ensure that if an employee withdraws a resignation (maybe they change mind), the system can cancel tasks and stop the offboarding. That could be an edge API – allowing HR to mark a resignation as retracted, which should then perhaps notify everyone and cancel tasks (or mark them void). Designing that is possible but may not be priority unless needed.

By formalizing the exit process, the system **tracks notice periods, ensures knowledge transfer, and asset returns**. This leads to smoother offboarding and ensures nothing is left incomplete when

someone leaves – improving compliance and saving HR time chasing these items. It also allows generation of documents like experience letters at a button-click, standardizing what used to be manual document work.

7. Task & Performance Tracking Service

Purpose: This module serves two related functions: general task management (assigning and tracking tasks to employees) and measuring performance, potentially by analyzing task completion and other metrics. It addresses the need for monitoring task progress, tracking hours, and ranking performance automatically. It can introduce a level of project/work management within the HRMS to ensure accountability and productivity.

Data Model: A new `performance` (or `tasks`) schema might include: - **Task:** Core table for tasks. Fields: `id, title, description, assigned_to (employee_id), assigned_by, date_assigned, due_date, status (Open/In Progress/Completed), completed_date, category, priority, etc., tenant_id`. We also include `estimated_hours` and `actual_hours` fields for time tracking on tasks. - **TaskLog/TimeEntry:** If we want detailed hours tracking, this table can log time spent on a task (e.g. an employee can log X hours on a task on a certain day). Alternatively, actual_hours could be directly set when marking complete if no incremental logging needed. - **TaskRelation:** Possibly to link tasks to other entities (for auto assignments). For example, a task might be linked to a specific Process like "Onboarding for Employee X" or "Exit for Employee Y" or a Project. We could have a generic `context_type` and `context_id` in task (e.g. context_type = Onboarding, context_id = that new employee's ID). This way, we know why the task was created and can filter tasks belonging to a process. - **PerformanceRecord/Score:** We might maintain a table summarizing an employee's performance metrics over a period (quarterly/yearly). Fields: `employee_id, period, tasks_completed, tasks_late, avg_completion_time, manager_rating, performance_score, rank`. This would be computed or updated periodically. - **PerformanceReview (optional):** If formal performance reviews are in scope, a table for manager ratings or 360 feedback could be included. Not explicitly in requirements, but might be implied by "performance ranking" that some input from managers or an algorithm is used.

Task Management: - *Assignment:* Tasks can be created manually by managers or automatically by the system in response to events. We'll cover automation soon. When a task is created, it is assigned to an employee with a due date. The assigned person gets a notification (and sees it in their ESS "My Tasks"). They work on it and update status or log hours. - *Tracking:* The assignee or the system updates the task status when done. Managers can view status of tasks they assigned or tasks in their team. The service can send reminders for tasks nearing or past due dates (e.g. daily check: any task due in 1 day or overdue -> send reminder to assignee and CC manager). - *Auto Assignment:* This is a key requirement – automatic task assignment means the system will generate tasks without manual intervention for certain scenarios: - **Onboarding:** as mentioned earlier, when a new employee is created, several onboarding tasks should spawn. The Recruitment/HR Core module would trigger this. Implementation: HR Core service, after adding a new employee, calls Task service API to create tasks (with context "Onboarding" and the new employee's ID). Templates for onboarding tasks can be defined (like a list of standard tasks for every new hire: IT setup, orientation scheduling, etc.). These tasks are assigned to relevant roles (IT, Manager, Buddy, etc.). We can maintain a template table for such processes. - **Exit:** similarly, on exit event, tasks for asset return, knowledge transfer are created (context "Exit"). - **Performance Improvement or Goals:** Possibly if a performance review flags an issue, HR could auto-assign training tasks or improvement plan tasks. - **General Recurring tasks:** The system could also auto-assign periodic tasks like "Submit timesheet every Friday" to employees, or "Complete mandatory training" to all (if integrated with training, though out of scope). The architecture for this could be a rule-based scheduler or simply trigger code at appropriate points as described. The Task module itself could

hold templates and have an API like `POST /tasks/generate?template=onboarding&employeeId=X` which creates the list. - *Hours Tracking:* If needed, employees can log hours spent on each task (like timesheets). This could be done via TaskLog entries or by updating actual_hours. Because we also have attendance which tracks total hours at work, this task-hours is more granular allocation of that time. We may use it for performance metrics (how effectively time is spent). - *Completion and Evaluation:* When tasks are completed, the system records actual completion date and actual hours. This data will be used to derive performance metrics such as: - On-time completion rate (% tasks completed on or before due date). - Average delay (if any tasks overdue). - Task completion count. - Possibly quality if tasks have a review step (not mentioned, but could be if manager reviews the work). - *Performance Ranking:* We can compute a performance score for each employee, perhaps monthly or quarterly, based on task data and other inputs: - A simple formula might be: score = (# tasks completed on time * some weight) + (total hours put in) + (manager rating if available) – (# tasks overdue * penalty). We could normalize this so that we can rank employees. The top performers according to this score can be highlighted. - Alternatively, if formal performance reviews exist, those scores can be combined. The requirement likely expects some automated ranking, possibly as a way to identify top performers (maybe to reward or just inform). We will implement a scheduled computation: e.g. at end of month, calculate each employee's performance score from tasks and store it in PerformanceRecord. Then perhaps rank them by department or overall in tenant. This data can be shown to managers or leadership. (Caution: in practice automated ranking can be sensitive; but since it's requested, we implement it as a feature that can be tuned or turned off per tenant if undesired).

APIs: - For tasks: - `POST /tasks` to create a new task (for managers or system internal calls). Fields include assignee, description, due date. - `GET /tasks?assignedTo=me&status=Open` (for employees to list their tasks), `GET /tasks?team=mine` (for a manager to list tasks of their subordinates, if needed). - `PUT /tasks/{id}` to update status (e.g. mark complete, update progress). - `POST /tasks/{id}/logtime` if implementing time logs (with hours and optional date). - For performance: - `GET /performance/score?period=2026-Q1` – for HR or managers to retrieve performance scores/rankings for that period. Possibly filter by department. - The ranking could also be part of this response or separate: `GET /performance/rankings?period=2026-Q1`. - If managers are to input ratings: `POST /performance/review` (with employee, rating, comments) – not explicitly required but could be extension. - If needed, `GET /performance/review/{employeeId}?period=...` to fetch details.

Integration: - As described, this module gets triggers from others to create tasks. We can implement it by having those modules call the Task API when needed. E.g. Recruitment service calls Task API when a candidate is hired (to create onboarding tasks), Exit calls when resignation accepted. This is a straightforward approach. Alternatively, using the event bus: recruitment publishes "EmployeeOnboarded" event, Task service subscribes and creates tasks. Either way ensures decoupling. Event-driven is cleaner but a direct API call is simpler to implement initially. - For performance, if we consider integrating manager's subjective feedback, that could come from outside the task system, but since not mentioned, we will focus on task-derived metrics. - The Attendance module could also supply data here indirectly: if needed, we might incorporate absenteeism or overtime into performance evaluation (some orgs consider attendance part of performance). But since tasks are main focus, we stick to tasks.

RBAC: All tasks are tenant-scoped. Assignment and visibility: - An employee sees tasks where they are the assignee. - A manager can see tasks they assigned *or* tasks of their direct reports (this may be needed to monitor team progress). We can implement that by checking if the assignee's manager_id = current user. Alternatively, tasks could have a field for "visible_to_manager" flag and we simply allow manager role to query tasks by team. - HR admins might see all tasks or at least have the ability to

query tasks for administrative processes. - Automatic tasks (like compliance tasks) could be assigned to specific roles (e.g. IT team user group). We might need a way to assign to a role or group rather than a specific user – perhaps assign to a pseudo-user representing a team, or create a task per responsible user.

Scalability: Task volumes could grow, but usually manageable (each employee might have a handful to dozens of tasks, not thousands). The service can scale horizontally if needed. It will frequently interact with users (listing tasks in ESS, updating statuses in real time), so needs to be responsive. Index by assignee, status for quick retrieval. Use websockets or push notifications if needed for instant update on task assignment (optional nice-to-have).

Impact on Performance: By systematically tracking tasks and their completion times, the system provides data to evaluate employee performance. The **performance ranking** feature will highlight top performers – e.g. “Employee X completed 95% of tasks on time with a high quality score” vs others. The HR team can use this alongside other evaluation methods. Because the ranking is automated, it **improves transparency** in how performance is measured (though it should be used carefully).

8. HR Chatbot Integration

Purpose: The HR Chatbot provides an AI-powered conversational interface for employees to query HR information or perform simple actions via chat. Our focus is on the backend APIs to support chatbot interactions, not the NLP engine itself (which may be a third-party service or a separate AI component). The chatbot should be able to retrieve information like leave balances, attendance, payroll details, and policy info in a Q&A format, based on secure access to the user’s data.

Design Considerations: The chatbot could be integrated into platforms like MS Teams, Slack, or the HR portal itself. Typically, a chatbot front-end will send the user’s query to a backend (either via a direct webhook or through an AI service that parses intent). We assume we have a layer that translates the user’s natural language question into a specific intent or API call. For example: - User asks: "How many vacation days do I have left?" -> The NLP interprets intent as `GetLeaveBalance` and perhaps identifies the user. - The backend then needs to answer: it should call the Leave service to get the exact number and then format a response.

To facilitate this, we introduce a **Chatbot Service** (as shown in the architecture diagram) which acts as an intermediary between the chatbot client and the rest of the HRMS APIs: - The Chatbot service receives a request containing the user’s ID (or some token) and the interpreted intent (like `LEAVE_BALANCE_QUERY`). - It authenticates/authorizes the request (ensuring it’s a valid session or chatbot token mapped to a user). - It then calls the appropriate internal API. This could be direct service calls or through the gateway. For example, for leave balance, it may call `LeaveService.getBalance(user)`; for last payslip, call `PayrollService.getPayslip(user, lastPeriod)`. - It then formulates a friendly reply text (and possibly rich content) to send back via the chatbot channel.

APIs & Intents: We can define a set of supported “intents” or direct endpoints: - **Leave Balance:** Maps to Leave service data. E.g. Chatbot API has an endpoint `/chatbot/leave-balance` which internally calls the Leave module. Or if using intent names, a single `/chatbot/query` endpoint could accept something like `{ intent: "LEAVE_BALANCE", user: X }`. - **Attendance Info:** e.g. "Did I check in today?" or "What's my attendance this month?" – the service can fetch today’s attendance or summary from Attendance service. - **Payslip/Increment:** e.g. "Show me my last salary slip" – chatbot service calls Payroll to get last slip (maybe it returns a summary or a link to PDF). Or "When is my next increment

due?" – this might check if there's an increment scheduled in Payroll for the user, or respond if none scheduled. - **HR Policies:** e.g. "What is the leave policy?" – for such informational questions, the answer might be stored in a knowledge base (could be a static FAQ in DMS or even just an HTML page). We could store common Q&A in a table or use an external FAQ service. Minimally, we could have a table `FAQ` with question patterns and answers. The Chatbot service, upon an intent like `POLICY_QUERY` with an entity like "leave policy", would look up the answer in that table or document store (DMS where policy PDFs are stored) and respond with a summary or link. - **Generic fallback:** If the chatbot is more AI-driven (like using an LLM that can be fed data), the Chatbot service might act as a proxy that passes user queries to an external AI service (like an OpenAI API) with some context. But since we focus on backend we assume most queries fall into predefined categories (attendance, leave, payroll, policy).

Security: "Secure Employee Access" is emphasized, meaning the chatbot should only provide data to authenticated users about their own info (or the info they are allowed to see). We must ensure the integration with chat platform passes the user's identity securely. There are a few approaches: - If the chatbot is integrated into the HR portal (user already logged in), it can use the same session/token – the chat messages can be sent to our server with the user's auth token. - If on external platforms (Teams/Slack), we might need a linking mechanism (e.g. user first verifies or logs in via OAuth, then the chatbot knows their HRMS account mapping). - The Chatbot service will verify the token or use a signed payload from the chat platform. Once verified, it knows the user's ID and tenant to use for queries. - All responses must obey RBAC: since most queries are personal (their own data), it's straightforward. If we allowed questions like "Who is my team's top performer?" then it's accessing others' data, but such is not explicitly mentioned. We likely stick to personal data and general HR info.

Logging & Monitoring: The Chatbot service should log all queries (perhaps sanitized if necessary) and the responses given. This helps improve the bot and also serve as an audit if the bot gives out incorrect info or needs to be reviewed. Given that AI is involved, maintaining logs is vital [8].

Integration Example: - User: "How many sick leaves have I used this year?" - NLP might return intent: `LEAVE_USAGE` with entity `sick leave`. - Chatbot service receives this, identifies user U and tenant T. - It calls Leave service: e.g. `GET /leave/usage?year=2026&user=U&type=Sick` which returns maybe "Used 2 out of 8". - Chatbot service formats: "You have used 2 sick leave days out of your 8 allotted for 2026. You have 6 remaining." - Returns this to the chat client.

Another: - User: "What's my take-home pay this month?" - Possibly intent `CURRENT_PAY` – it calls Payroll for the latest pay record of user U, gets net pay, and responds "Your take-home pay for Feb 2026 is Rs X (after all deductions)."

If the question is something the bot can't handle or is very complex, the service might either integrate with an AI Q&A system or respond with a fallback ("I'm sorry, I can't help with that. Please contact HR."). We can also configure escalation: e.g. if user asks something not recognized, open a ticket or inform an HR rep to follow up.

Scalability: Chatbot queries might be frequent but they are light (read operations mostly). The Chatbot service is stateless and can scale out to handle many concurrent queries. Each query fan-outs to maybe one or two internal service calls, which are also lightweight if indexed. We should watch for any heavy questions (like "Show me all my payslips of last 5 years" – which could be a lot of data, but still manageable). If using an external NLP API, that might have rate limiting or cost, but that's outside our system's scope.

Conclusion on Chatbot: The chatbot integration will leverage all the modules we built, providing a modern conversational interface. It essentially repurposes our API capabilities in a user-friendly Q&A form. This aligns with the "AI Powered HR Chatbot" requirement – our backend ensures the chatbot has access to up-to-date info on leave, attendance, payroll, etc., all while maintaining security. Over time, as more Q&A pairs are identified, we can expand the chatbot's knowledge base (for example, add answers for "What is the HR email?" or "How do I reset my portal password?" etc., possibly connecting to IAM or static info). Because the backend is modular, adding a new query type usually means just calling another service's API, which is straightforward.

Cross-Cutting Concerns and Best Practices

Beyond the individual modules, certain design aspects apply system-wide to ensure the platform is enterprise-grade:

1. Unified Authentication & Authorization:
All services rely on the central IAM module for authenticating users and enforcing roles/permissions. We will likely implement a JWT-based auth system: users log in and receive a token that includes their user ID, tenant ID, and roles. The API Gateway will validate this token on each request (signature and expiration) and extract the user info. It can then pass on the user identity (via headers or inside the request) to the downstream services. Each service additionally performs authorization checks relevant to the action (e.g. ensuring the user has `ROLE_HR` for an HR-only endpoint, or that `userId` in the request matches token for self-service endpoints). This layered approach (gateway + service checks) adds security.

We must update the IAM data for new roles introduced: e.g. roles for Recruitment, Payroll, etc., and map them to permission scopes. The RBAC model will be designed so that roles are **tenant-scoped** [2]. This means, if a user has the Recruiter role, it is tied to a specific tenant/company – the system will always check that context to avoid any leakage. The WorkOS model of tenant-scoped roles can be implemented by including the tenant in user-role assignments [7]. Our IAM schema likely already has a structure for that (like a junction table of user, tenant, role).

For flexibility, we may allow custom roles per tenant if needed (some enterprises might want a slightly different division of duties) – our design can support that as the RBAC checks can be data-driven (like checking permissions rather than hardcoding role names). However, initially a fixed global role set, reused per tenant, keeps things simple [12].

2. Multitenancy Enforcement:
All services are built to be multi-tenant from the ground up. Each database table has a tenant identifier, and ORMs or queries will always include tenant filters. To avoid any chance of cross-tenant data mix, we could implement: - A **tenant context** that's set at each request based on the authenticated user's tenant. The service methods then automatically apply that context to all DB operations (some frameworks support a "current tenant" context to auto-filter queries). - Additional safe checks in the code at boundaries (e.g. if a user tries to access an object by ID, the service verifies that object's tenant_id == user's tenant_id, otherwise returns not found or error).

This way, even if a programmer mistakenly forgets a filter, the system won't return another tenant's data because of these safeguards [3]. We treat tenant isolation as a critical security property.

We also isolate configuration per tenant – for example, each tenant can have its own leave policy, holiday calendar, payroll settings, etc., stored with tenant_id in config tables. This allows a high degree

of customization without affecting other tenants [13] (which is important for a SaaS HRMS that serves multiple clients). It increases the complexity of testing all scenarios, but it provides flexibility needed for enterprise deals.

3. Logging, Monitoring, and Traceability:
Implement comprehensive logging in each service. Every request should log key details (request path, user, tenant, parameters) and the outcome (success/failure and time taken). These logs will be centralized (e.g. shipped to an ELK stack or a cloud logging service) for analysis. We will include a **Correlation ID** (e.g. a UUID) for each external request [8]. The API Gateway can generate this ID for each incoming API call (or accept one if the client passes it), then forward it as a header to all services. Each service includes it in their logs. This allows us to trace a single user action through multiple microservices easily. For example, approving a leave might involve Leave service and Notification service; with a correlation ID, we can open logs and see the entire sequence across services with that ID.

We will also capture error logs and exceptions with enough detail to debug but without exposing sensitive data. A centralized **error tracking** system (like Sentry or Application Insights) can alert developers to any unhandled exceptions in production.

Monitoring: Each service will expose health endpoints and metrics. We can use tools like Prometheus to scrape metrics from services (e.g. request counts, latency, DB query time, queue lengths). Setting up dashboards (Grafana, etc.) helps ops to spot performance issues or failures. For instance, monitor how long payroll jobs take, or how many pending tasks remain in the queue, etc.

4. Asynchronous Processing & Scheduling:
We have incorporated a message queue for decoupling heavy tasks. We will use a reliable message broker (RabbitMQ, AWS SQS, etc.) with persistence, so tasks aren't lost if a service restarts. Each task type will be processed by dedicated **worker processes** (which can be part of the service or separate microservices). For example, a "cv_screening" queue could be handled by a ML service worker, whereas a "payroll_run" queue handled by the payroll service's workers. By segregating, we ensure one slow consumer doesn't block others and we can scale each independently (e.g. spawn more CV processing workers if recruitment load is high).

For scheduling, we might integrate a distributed scheduler or simply use cron jobs on one instance of a service (with leader election if multiple). E.g. the Attendance service can perform daily checks at midnight, the Payroll service can schedule monthly runs or remind if not run by certain date. Alternatively, a central scheduling service could trigger these via APIs (some companies use tools like Quartz jobs in Spring or Hangfire in .NET, etc., or Kubernetes CronJobs to hit an API on schedule). The exact mechanism can be chosen based on tech stack. Key is to avoid duplicate triggers in a cluster (so ensure only one node runs the scheduled task, perhaps by using a DB lock or designate one instance as the scheduler).

5. Resilience and Fault Tolerance:
We aim for no single point of failure. The database is a single instance logically, so we will use PostgreSQL high-availability configuration (e.g. primary-replica with failover) to ensure uptime. Similarly, the message broker should be clustered or use a managed service that is redundant.

For the microservices, running multiple instances behind load balancer means if one instance goes down, others continue serving. We will implement **circuit breakers** on any synchronous inter-service calls to prevent cascading failures. For example, if Payroll service calls Attendance and Attendance is down or slow, the Payroll service should timeout after a short period and handle it (maybe by falling

back to cached data or returning an error rather than hanging). Tools or libraries can help implement this.

We will also use retries with backoff for transient failures. If Notification service fails to send an email due to a network glitch, it can retry a couple times. If a background job fails, it can requeue or log for manual intervention depending on criticality.

Data integrity and transactions: Within a single service, use DB transactions to keep data consistent. Across services, since distributed transactions (2PC) are not practical at our scale, we rely on eventual consistency via events. For instance, when hiring a candidate, if creating user in IAM fails while HR Core succeeded, our design should detect and recover: maybe the Recruitment service will log a compensation or show an error to retry creating the user. Or better, orchestrate that with a saga pattern if complexity grows – e.g. have a Orchestrator (like workflow) that confirms all steps or rolls back (like if user creation fails, mark employee record as inactive or remove it). Given the relatively simple flows, careful ordering and retries suffice.

6. DevOps and Deployment:
We recommend containerizing each service (Docker) and using an orchestration platform like Kubernetes or a cloud PaaS to deploy. This makes scaling and updating easier. Use CI/CD pipelines so that new code for a service is automatically tested and deployed with minimal downtime. Implement blue-green or canary deployments for critical services (like IAM, Payroll) so we can monitor a new release with a fraction of traffic before full rollout.

Each microservice will have its own repository or at least separate build artifact, reflecting the modular approach. They can share common libraries for things like logging, auth client, etc., to avoid duplication and enforce consistency (like the **Nmbrs "fabric" approach of reusing building blocks** [14]).

7. Audit and Compliance:
We should maintain audit logs for key actions, stored in a tamper-evident way. E.g. any time an HR or manager changes data (approves leave, changes salary, etc.), record who did it and when. This could be in each module's tables (like adding `updated_by` and `updated_at` fields, or separate audit tables). This is important for compliance (e.g. to comply with financial audit requirements for payroll, or just internal accountability). Also, ensure personal data protection by limiting who can access sensitive info (like hide or mask SSN equivalents in logs, etc., as needed for privacy laws).

8. Testing and Quality Assurance:
With many integrated modules, testing is crucial. We will implement automated tests for each service (unit and integration tests) and end-to-end tests for key user scenarios (like an employee applying leave and manager approving, resulting in balance change and email notification). Using test automation ensures stability as we enhance features.

9. Extensibility:
Our architecture is set up to handle new modules in the future easily. For example, if we later add a **Training/LMS module** or **Performance Appraisals** or **Benefits management**, we can plug it in as another service, add a schema, and integrate via the same patterns (API calls/events). The gateway can route requests to it and IAM can manage its roles. This modular approach thus "future proofs" the HRMS to an extent, allowing continuous innovation with minimal disruption [15] .

Implementation Roadmap and Rollout Plan

Implementing this comprehensive HRMS backend will be done in phases to manage complexity and ensure stability. A staged rollout allows testing each module thoroughly and training users step by step. Below is a recommended phase-wise plan:

1. **Foundation and Infrastructure Setup:**
2. Establish the **API Gateway** and common services (IAM, Tenancy, Notification) if not already robust. Enhance IAM for any needed features (e.g. JWT issuance, new roles for upcoming modules). Set up the **Message Queue** system and integrate basic logging/monitoring infrastructure (correlation ID mechanism, central log aggregator) [8].
3. Implement the Notification/Email Service early, since many modules will use it for sending emails. This service can start as a simple SMTP sender with a queue subscription.
4. Prepare development and staging environments with CI/CD. Define coding standards and ensure all teams use the shared libraries for things like auth and logging (to maintain uniformity).

5. **Data migration:** If any existing data (e.g. current employee records in `hr_core` or user accounts in `iam`) needs cleaning or refactoring to fit new schemas, handle this now. For example, if `hr_core` lacks a field for manager or dept needed for leave approvals, add it before rolling out leave module.

6. **Phase 1 – Recruitment & Onboarding Module:**

7. Develop the Recruitment/ATS service and schema. Focus on the core hiring workflow (job posting to candidate hire). Integrate it with IAM (for creating user on hire) and HR Core (for creating employee records). Use dummy or basic AI for CV screening at first (could even be a simple keyword match or a stub that marks all as medium fit), unless an AI model is readily available.
8. Roll out Recruitment module to HR department users first. They can start posting jobs and tracking applicants internally. Run it in parallel with their old hiring method for a pilot.
9. Test the end-to-end scenario: create a job, apply as a candidate (maybe by HR playing role of candidate or a test site), screen, schedule an interview, mark hired, verify that an Employee record and IAM account is created properly. Adjust as needed.

10. Provide training to recruiters/HR on using the new recruitment interface. Since this is mostly an internal module (not directly all employees), the user group is small, making iteration easier.

11. **Phase 2 – Core HR & Self-Service basics:**

12. At this stage, ensure the **HR Core** data is complete and accurate because upcoming modules depend on it (especially manager relationships, departments, etc.). It might involve migrating any missing data (like setting each employee's manager in the system).
13. Launch the **Employee Self-Service Portal** with a few basic features first, to get users on board. For example, allow employees to log in (via IAM) and view their profile, org chart, and maybe documents. This tests the waters of all employees using the system.
14. On the backend, this means exposing some HR Core APIs (profile info, org structure) and DMS APIs (so they can view payslips or policy documents as we upload them). You might generate initial content such as posting company HR policies in DMS and making them accessible.

15. The UI/UX rollout here is critical: communicate to employees how to log in and use the portal for these basic functions. Solicit feedback and ensure authentication, role assignments, etc. are working smoothly at scale.

16. This also serves as a test of system load with many concurrent users (if company is large). Monitor performance and tune if needed (e.g. maybe add caching for frequent profile picture loads or such).

17. **Phase 3 – Attendance & Leave Management:**

18. Implement the Attendance service and Leave service together, since they often go hand-in-hand for end-users. Develop the leave request workflow (integrating with Workflow or using in-service logic). Also, integrate attendance capture methods.
19. Pilot the **Attendance module** perhaps with a subset of employees or one office location – especially if integrating with a new device or using self check-in. Work out any issues in capturing data (for instance, if using web check-in, ensure users understand how to use it; if pulling from a biometric device, do a parallel run to verify data correctness).
20. Launch the **Leave management** to all employees for a specific leave type first (e.g. they can apply for casual leave through the system, other types still manual), or launch fully but instruct managers on the new approval process. Often companies might want to test with HR or one department before company-wide. For example, HR could simulate requests or use a small group to see if notifications and approvals flow correctly.
21. Once confident, turn on the leave module for all employees. This likely requires a **cut-over**: e.g. if they had an existing way to track leave (like Excel or legacy system), decide from which date the new system is authoritative. Possibly migrate current leave balances into the system (e.g. load each employee's remaining vacation days for the year into LeaveBalance table so the system starts with correct data).
22. Ensure all managers and HR are trained on approving leave in the system. Use internal memos or brief training sessions to get everyone on board. Emphasize the benefits (automatic updates, no emails needed, etc.).

23. Monitor carefully the first month of attendance & leave usage. There may be edge cases (e.g. someone forgets to clock out – HR needs to know how to adjust, or a leave request stuck because manager didn't act – HR needs ability to override, etc.). Use this feedback to add any needed admin features or refine the workflow.

24. **Phase 4 – Payroll Automation:**

25. This is one of the most sensitive modules. Develop the Payroll service fully, but likely **do not immediately use it for actual payroll processing** until thoroughly tested. First, for one cycle, run the payroll module in **parallel** with existing payroll methods. For example, during the next month's payroll:
    - HR runs the payroll through the new system (with test flag, not actually paying out).
    - Compare the generated results (each employee's net pay, deductions) with the results from the current method (be it manual or another software).
    - Investigate any discrepancies. Some differences may come from rounding or minor configuration issues (e.g. tax calculation differences). Adjust rules/config in the system as needed to match expected results.
26. Once the calculations consistently match and compliance is verified, plan a switch. Likely choose a less critical month or quarter for first live run. Have a rollback plan (for instance, if something fails on payday, be ready to calculate manually or revert to old method for that cycle).

27. During rollout, limit access to payroll module to only payroll staff. They should practice generating payslips, and maybe even run a small subset (like just one department's payroll fully on the system as a pilot).
28. After a successful pilot, officially adopt the payroll system for all employees. On rollout, ensure:
    - All employees' salary structures are entered correctly in the system beforehand.
    - Starting YTD (Year-to-date) figures for tax deductions, etc., are loaded if mid-year (to correctly calculate taxes for remainder of year).
    - Communicate to employees that their payslips will now come from the new system (maybe via ESS portal or email). Possibly do a demonstration on how to download their slip from ESS.
29. Keep a close eye on the first couple of live runs. Provide support for any issues (e.g. if an employee's tax seems wrong, double-check configuration).

30. Also implement the increment cycle management around appraisal time if possible. That might coincide with performance module rollout or could be separate. When increments are decided, HR can input them into system and generate letters.

31. **Phase 5 – Performance & Task Module:**

32. Introduce the Task management system. Possibly start by using it for **internal HR tasks first** (like those onboarding/offboarding tasks). HR and managers will see those tasks being assigned and completed, which familiarizes them with the concept.
33. Then encourage managers to use it for assigning work tasks or goals to their team. This might require some culture change if they're not used to logging tasks. So, provide guidance on how this can help track work progress.
34. Implement performance metrics gradually. Perhaps initially just display basic stats (how many tasks each person completed) in the manager's view. After a period of data collection, introduce the performance ranking feature.
35. We might avoid showing a "rank" outright at first, and instead share performance reports with management until it's validated. Thereafter, it could be made visible in ESS (maybe to the employee themselves or managers).
36. Integration: ensure by now that onboarding and exit processes are creating tasks automatically. Check that those tasks close properly (like if an employee's tasks remain open after leaving, maybe HR closes them or system auto closes on last day – refine logic as needed).
37. The chatbot (next phase) will also benefit from tasks data (e.g. an employee might ask "Do I have any pending tasks?" – which the chatbot could handle if we wire it up).

38. This module is less critical than payroll, so a lighter rollout is fine, but do manage expectations about the automated performance scoring – HR should calibrate if needed (for example, if the algorithm needs tweaking, do so with HR feedback).

39. **Phase 6 – HR Chatbot Integration:**

40. Finally, deploy the Chatbot service and interface. Since it relies on all other modules being live and populated, it comes last. Begin with a **limited launch or beta**:
    - Maybe enable the chatbot to a small group (like only HR team or a specific department) to test common queries.
    - Fine-tune the NLP intents if using an AI service – see what questions people actually ask and adjust the chatbot's understanding.

    - Ensure the chatbot's answers are correct and not divulging anything sensitive inadvertently (especially if using AI, ensure it doesn't hallucinate answers that are wrong or break confidentiality).
41. Gradually roll out chatbot to all employees as a feature in the portal or on their work chat platform. Advertise what it can do ("Try asking the HR Chatbot about your leave balance or policy questions!") to drive adoption.
42. Continuously monitor chatbot logs for unhandled queries. Use that to expand its capabilities (this is more of an AI training task).
43. The chatbot is a nice-to-have, so if any issue arises, one can temporarily disable it without affecting core HR operations.
44. **Ongoing and Future Work:**
45. After all modules are live, gather user feedback and metrics. For example, see if leave approval times improved, if HR is spending less time on payroll, etc. Use those to demonstrate the **business benefits** realized: a fully automated HR lifecycle, time saved, error reduction, greater transparency (employees have self-service visibility) [16].
46. Plan for enhancements: perhaps add more AI, such as predictive analytics (e.g. predict attrition via performance and engagement data), or add complementary modules like Training or Travel management if needed. The modular architecture will accommodate these easily by adding new services without disrupting existing ones [4].
47. Regularly update compliance rules (tax rates, etc.) in the system as needed, and keep security up to date (e.g. periodic pen-tests, dependency updates) to maintain enterprise-grade robustness.

Throughout the rollout, **change management** is crucial. This means communicating with all stakeholders (employees, managers, HR staff) about new features and processes, and ensuring they are comfortable using them. The staged approach allows each group to adapt gradually rather than all at once. By Phase 6, the organization should be fully transitioned to the new HRMS backend, leveraging all its features integrated on a solid, scalable architecture.

By following this design and rollout plan, the company will have a scalable, multi-tenant HRMS backend that cleanly separates concerns across modules yet provides a seamless experience through the unified portal and integrated processes. The system will be capable of handling the entire employee lifecycle **from recruitment to retirement** with automation at every step, delivering on the goals of saving HR time, reducing errors, improving transparency, and centralizing HR control. Moreover, the architecture's modularity and use of best practices (logging, auth, async, etc.) ensure that the solution is **enterprise-ready – stable, secure, and extensible for future needs** [17]. Each module can scale independently as usage grows, and new features can be integrated with minimal friction due to the standardized service interfaces [1]. This design sets the stage for continuous improvement of the HR platform, including incorporating emerging technologies (AI, analytics) as needed, keeping the HRMS on the cutting edge while maintaining reliability for daily operations.

---

[1] [4] [5] [15] Benefits of Adopting Modular Technology for HR - PeopleSpheres
https://peoplespheres.com/benefits-of-adopting-modular-technology-for-hr/

[2] [3] [7] [12] How to design an RBAC model for multi-tenant SaaS — WorkOS
https://workos.com/blog/how-to-design-multi-tenant-rbac-saas

[6] Choose the right PostgreSQL data access pattern for your SaaS ...
https://aws.amazon.com/blogs/database/choose-the-right-postgresql-data-access-pattern-for-your-saas-application/

[8] [14] HR Workflow — Developing the structure of the first microservice | by Visma Nmbrs | Nmbrs Tech Blog | Medium
https://medium.com/nmbrs-tech-blog/hr-workflow-developing-the-structure-of-the-first-microservice-f60519e58551

[9] [11] [13] [17] Multitenant HRMS Product Development using Microservices
https://www.indiumsoftware.com/casestudies/multitenant-hrms-product-development-using-microservices.pdf

[10] PostgreSQL's schemas for multi-tenant applications - Stack Overflow
https://stackoverflow.com/questions/44524364/postgresqls-schemas-for-multi-tenant-applications

[16] HR_Management_System_Demo_Content.docx
file://file_000000002e1c720899e9e038d524618c